

项目报告

本项目是一个以信科的吉祥物“攻城狮”为主角的一个桌面宠物程序

程序功能介绍

- 实现了“攻城狮”的各种不同的动作形态
- 实现了ai的API的配置
- 实现了文件传输系统

各模块和类的设计细节

模块一：桌面宠物整体框架

类设计

类名	继承	职责
PetWindow	QWidget	主窗口：无边框透明置顶窗口，统合所有子模块，处理鼠标拖拽、右键菜单、文件拖放
PetController	QObject	有限状态机：驱动帧动画、随机动作调度（5s间隔）、强制状态锁定、拖拽态保存/恢复
AnimationManager	(无)	帧数据管理：从 资源目录加载各状态的 PNG 帧序列，提供 frame/interval 查询
TrayMenuManager	QObject	系统托盘：托盘图标 + 右键菜单（Show/Hide、AI开关、状态控制、退出），通过信号与 PetWindow 通信

核心技术点

- **窗口:** `FramelessWindowHint | Tool | WindowStaysOnTopHint + WA_TranslucentBackground`，通过 `QPainter::drawPixmap` 逐帧渲染。
- **状态机:** 7 种状态（IDLE/DRAGGED/WALKING/SLEEPING/WORKING/CELEBRATING/SAD），两个 `QTimer` 分别驱动帧更新（频率按状态可变）和随机动作（固定 5s）。
- **拖拽:** `mousePressEvent` 记录全局坐标偏移，移动超过系统阈值后进入 DRAGGED 态；释放时恢复原态。
- **随机行走:** 在屏幕可用区域内随机选目标点，沿直线以 `4px/frame` 速度插值移动。
- **睡眠循环:** 默认 3 次动画循环后自动回到 IDLE。

- **强制状态:** `setForcedState()` 锁定状态，阻止随机动作调度，通过 `clearForcedState()` 解锁。

模块二：AI API 调用

技术栈: Qt Network (`QNetworkAccessManager`) + OpenAI 兼容 API (默认 DeepSeek)

类设计

类名	继承	职责
AIChatManager	<code>QObject</code>	核心：API 配置管理、对话历史维护、HTTP 请求/响应处理、 <code>QSettings</code> 持久化
AISettingsDialog	<code>QDialog</code>	设置 UI：enabled / API Key / endpoint / model / systemPrompt / temperature 表单编辑
ChatBubbleDialog	<code>QDialog</code>	气泡 UI：Input 模式（用户输入文本）和 Display 模式（展示 AI 回复，6.5s 自动关闭）

核心技术点

- **协议:** POST `{endpointUrl}`，Header 为 `Authorization: Bearer {apiKey}`，body 为 OpenAI Chat Completions 格式 JSON。
- **对话历史:** `QList<ChatMessage{role, content}>`，每次请求前 `buildMessagesPayload()` 序列化；`historyLimit` 限制上下文长度（默认 10 条），`trimHistory()` 从头裁剪。
- **API Key 存储:** Base64 编码 + `b64`：前缀后存入 `QSettings`（注册表），读取时解码还原（混淆级保护）。
- **错误处理:** 按 HTTP 状态码（401/429/5xx）分类给出中文错误提示，尝试解析响应体 `error.message`。
- **异步回调:** `QNetworkReply::finished` → lambda 解析 JSON → 提取 `choices[0].message.content` → emit `replyReady()`。
- **气泡定位:** 无边框异形对话框，背景用 `thoughtbubble.png` / `bubble.png`，定位在宠物右上角并自动修正屏幕边界。

模块三：文件传输功能

技术栈: `cpp-httplib 0.48.0`（HTTP 服务器）+ `libqrencode 4.1.1`（二维码）+ Qt 资源内嵌 HTML 前端

类设计

类名	继承	职责
FileTransferServer	QObject	HTTP 服务器：独立 QThread 中运行 cpp-http lib Server，提供 / 页面、/list API、/download 流式接口
FileTransferDialog	QDialog	传输窗口：展示二维码、IPv6 URL、文件状态、停止按钮
QRCodeGenerator	(静态类)	二维码：封装 libqrencode，generateQRCode(text, size) → QPixmap

核心技术点

- **线程模型:** moveToThread(&m_workerThread) 将服务器移至工作线程；QAtomicInt 跨线程查询状态；m_svr->stop() + quit() + wait(3000) 安全停止。
- **IPv6 获取:** 遍历 QNetworkInterface::allInterfaces()，过滤虚拟接口 (virtual/vmnet/docker 等) 和链路本地/组播/ULA 地址，返回全球公网 IPv6。
- **安全机制:**
 - UUID token 认证 (所有请求须带 ?token=xxx)。
 - resolveFilePath() 仅根据文件名在 m_sharedFiles 列表中匹配完整路径，防止路径遍历。
 - QMutex m_filesMutex 保护文件列表并发访问。
- **API 路由:**
 - GET / → 返回 file_browser.html (替换 {{TOKEN}} 模板占位符)。
 - GET /list → 返回 JSON [{name, size, mtime}]。
 - GET /download?file=xxx → 流式传输，64KB 缓冲分块，Content-Disposition: attachment。
- **动态添加:** 服务器运行期间 addFiles() 线程安全追加文件，手机刷新页面即可。
- **二维码:** QRcode_encodeString8bit() → 逐模块 QPainter 绘制 → QPixmap，纠错级别 M。

项目分工

- 李子政主要完成了这个项目整体的大的框架，完成了通过图片来表示不同动作的功能，随着时间定时在不同的动作之间进行切换的功能，完成了部分美术素材的创作
- 段治瀛完成了支持桌面宠物在default模式下进行随机游走的功能，部分美术素材的创作，完成了关于ai的API的配置，完成了关于文件传输系统的制作

项目总结和反思

- 明确分工：提前确定好各自的职责与交付的deadline，避免工作重复遗漏
- 保持同步：组内保持交流，多次提交且每次提交之后都进行merge操作，确保项目能够正常运行
- 标好注释：标好注释标好各个函数，类的相应接口和功能，提高效率，节省时间
- 做项目保持专注：在添加某一个功能或者函数的时候最好一次性完成，不要拖拉。